

Institutional Fraud Detection Engine

Enterprise Technical Blueprint & Interview Preparation Guide

SECTION 1: EXECUTIVE SUMMARY & SYSTEM ARCHITECTURE

This document outlines the end-to-end architecture, design patterns, security controls, and deployment pipelines for a production-grade, low-latency transaction fraud scoring system. Built for financial institutions, the system combines machine learning probabilistic inference with deterministic regulatory business rules.

System Architecture Overview

The engine operates on a decoupled microservice pattern containerized via Docker and orchestrated with Docker Compose. It separates the stateless scoring inference API from the analytical monitoring dashboard while preserving strict ACID-compliant audit trails.

Component	Technology	Core Responsibility
Scoring Gateway	FastAPI (Python 3.10)	Low-latency REST API, request validation, authentication, ML inference.
Inference Engine	XGBoost & Scikit-Learn	Probabilistic risk scoring based on categorical and velocity features.
Monitoring Portal	Streamlit	Real-time operator testing UI and regulatory audit log viewer.
Audit Store	SQLite (Embedded)	Immutable transactional decision logging for AML compliance.
Orchestration	Docker & Compose	Isolated multi-container runtime environment and networking.

Interview Talking Point: "We decoupled the scoring API from the UI dashboard using Docker containerization. This ensures that scaling or crashing the visualization layer has zero performance impact on the core sub-millisecond scoring pipeline."

Key Engineering Pillars

- Dual-Layer Risk Evaluation:** Combines machine learning probability vectors with hard-stop regulatory compliance gates.
- Cryptographic API Security:** Header-based authentication (`x-api-key`) mapped to authorized corporate entities.
- Stateful Audit Logging:** Every decision is recorded with timestamps, risk thresholds, and institution markers.

1. Feature Engineering & Schema Validation

Incoming transactional payloads are validated at the API boundary using Pydantic models. Numerical features (amount, distance, velocity) and categorical features (merchant category) are verified for type safety before encoding.

Feature Name	Data Type	Description
transaction_id	String	Unique identifier for ledger tracking.
amount	Float	Monetary value in USD. Triggers AML rules if $\geq \$50,000$.
velocity_1h	Integer	Transaction frequency in the past 60 minutes.
merchant_category	Categorical	One-hot encoded (retail, travel, electronics, etc.).

2. Model Loading & Lifecycle Management

Using FastAPI's lifespan asynchronous context manager, serialized XGBoost models (fraud_model.pkl) are loaded into system memory exactly once during application startup. This prevents disk I/O bottlenecks during high-frequency API calls.

Code Snippet Pattern (Lifespan Context):

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    init_audit_db()
    ml_artifacts['model'] = joblib.load(...)
```

3. Feature Alignment & Inference Execution

To prevent dimensional mismatch between inference requests and training artifacts, incoming categorical variables undergo one-hot encoding, followed by strict alignment against expected feature matrices, filling missing columns with zeros.

- **Probability Thresholds:** Scores below 0.35 yield APPROVE; 0.35 to 0.70 yield CHALLENGE_2FA; scores ≥ 0.70 yield BLOCK.
- **Cold-Start Safety:** If model binaries are missing at startup, the service raises a fatal `RuntimeError` rather than failing silently.

The Need for Deterministic Guardrails

While machine learning models capture non-linear fraud patterns, probabilistic models alone are insufficient for institutional Anti-Money Laundering (AML) mandates. Regulators require strict deterministic boundaries for extreme transactions.

Implemented Business Rules

Condition / Trigger	Enforced Decision	Risk Score Adjustment	Compliance Context
Amount $\geq$ \$50,000 AML	BLOCK	Forced to $\geq$ 0.95	Large-scale transfer safety net; prevents blind ML approval.
Velocity 1h $\geq$ 8 Velocity	BLOCK	Forced to $\geq$ 0.95	Card-testing automated bot attack mitigation.
Amount $\geq$ \$20,000 or Velocity 1h $\geq$ 4	CHALLENGE_2FA	Forced to $\geq$ 0.75	Enhanced due diligence / step-up verification required.

Interview Talking Point: "We layered deterministic business rules directly over our ML output. If a transaction is over \$50k, the model's statistical confidence is overridden by compliance law, forcing an immediate block and logging an immutable audit record."

Immutable Regulatory Audit Logging

Every decision passes through an SQLite audit logger. The database records the institution name, transaction ID, cardholder ID, evaluated risk probability, final decision, and UTC timestamp, ensuring complete non-repudiation for auditing.

1. Docker & Docker Compose Isolation

The system is packaged into two lean containers using `python:3.10-slim` base images. Docker Compose manages an isolated bridge network allowing containers to communicate securely via internal service names.

- **API Container (`institutional_fraud_api`):** Exposes port 8000. Internal communication uses hostname `http://fraud-api:8000`.
- **Dashboard Container (`institutional_fraud_dashboard`):** Exposes port 8501 for operator access.

2. Solving Container Networking Challenges

When running Streamlit inside Docker, browser-based requests to `127.0.0.1:8000` fail because `localhost` points inside the container itself. The correct architecture uses Docker's internal DNS resolution (`http://fraud-api:8000`).

3. Public Cloud Publishing Strategy

Component	Recommended Platform	Deployment Target
FastAPI Backend	Render / Railway	Web Service (Docker runtime, port 8000 exposed).
Streamlit Dashboard	Streamlit Community Cloud	Managed GitHub sync linked to public cloud API endpoint.

Interview Wrap-Up: "By containerizing our FastAPI backend and Streamlit dashboard with Docker Compose, we eliminated 'it works on my machine' issues, ensured reproducible builds, and established a clean pathway for multi-cloud enterprise production."